TH-4: ο μετρητής με καθυστέρηση

Ο 4-bit μικροελεγκτής σου (έκδοση 1.2, με timer) εκτελεί το `prog.asm`: μετράει 0, 1, 2 … 15, 0, 1 … στη θύρα εξόδου, με έναν βρόχο καθυστέρησης ανάμεσα. Οι τιμές είναι ίδιες με αυτές που βγάζει η προσομοίωση (`vvp run`).

1. Τι έχει μέσα ο TH-4

Κομμάτι	Μέγεθος	Τι κάνει
`A`	4 bits (0–15)	ο βασικός register· εδώ γίνονται οι πράξεις
`B`	4 bits (0–15)	δεύτερος register· μετρητής για βρόχους
`Z`	1 bit	flag: γίνεται 1 όταν ένα αποτέλεσμα είναι 0
`PC`	8 bits	η διεύθυνση της εντολής που εκτελείται
`RAM`	16 × 4 bits	16 θέσεις για μεταβλητές
`port_out`	4 bits	καταχωρητής της θύρας εξόδου (πόδια uio[7:4])
`port_in`	4 πόδια	η θύρα εισόδου (πόδια uio[3:0])
`timer`	25 bits	μετράει παλμούς ρολογιού μόνο όσο εκτελείται ένα WAIT· αλλιώς είναι σταματημένος, για οικονομία ρεύματος

Πώς είναι μια εντολή (1 byte)
`[7:4] = opcode  [3:0] = n`
Παράδειγμα: `ADD 3` = opcode 1, n 3 = `13` (hex).
Τα `JUMP`, `JZ`, `JNZ` πιάνουν **2 bytes**: το δεύτερο είναι η διεύθυνση.
Το opcode F είναι `TAB` όταν n = 0 και `WAIT n` όταν n = 1…15.

Αυτό το πρόγραμμα δεν διαβάζει τη θύρα εισόδου. Χρησιμοποιεί μόνο τους A, B και τη θύρα εξόδου.

2. Οι εντολές

Hex	Εντολή	Τι κάνει	Αλλάζει το Z	Hex	Εντολή	Τι κάνει	Αλλάζει το Z
`0n`	`LOAD n`	A ← n	✔	`9n`	`LOADM n`	A ← RAM[n]	✔
`1n`	`ADD n`	A ← A + n	✔	`An`	`LDB n`	B ← n	
`2n`	`SUB n`	A ← A − n	✔	`B0`	`DECB`	B ← B − 1	✔
`3n`	`AND n`	A ← A AND n	✔	`C0 aa`	`JNZ aa`	αν Z = 0: PC ← aa	
`40`	`IN`	A ← θύρα εισόδου	✔	`Dn`	`ADDM n`	A ← A + RAM[n]	✔
`50`	`OUT`	θύρα εξόδου ← A		`En`	`SUBM n`	A ← A − RAM[n]	✔
`60 aa`	`JUMP aa`	PC ← aa (πάντα)		`F0`	`TAB`	B ← A	
`70 aa`	`JZ aa`	αν Z = 1: PC ← aa		`F1–FF`	`WAIT n`	περίμενε τον timer (n = 1…15)	
`8n`	`STORE n`	RAM[n] ← A					

3. Το πρόγραμμα στη μνήμη

Αριστερά η διεύθυνση και τα bytes που γράφονται στη μνήμη προγράμματος, δεξιά η assembly.

Διεύθ.	Bytes	Ετικέτα	Εντολή	Σχόλιο
00	00		LOAD 0	
01	50	loop	OUT	γράψε τον A στη θύρα εξόδου
02	AF		LDB 15	καθυστέρηση: ο B μετράει 15 -> 0
03	B0	wait	DECB	
04	C0 03		JNZ wait	
06	11		ADD 1	επόμενο νούμερο
07	60 01		JUMP loop	

4. Η εκτέλεση, βήμα προς βήμα

Κάθε γραμμή δείχνει τις τιμές **μετά** την εκτέλεση της εντολής.

πορτοκαλί η τιμή άλλαξε σε αυτό το βήμα κίτρινη γραμμή άλλαξε η θύρα εξόδου κόκκινο κείμενο έγινε άλμα

#	PC	Εντολή	A	B	Z	OUT	Τι έγινε
1	00	LOAD 0	0	0	**1**	0000	Βάζει τον αριθμό 0 στον A. Το αποτέλεσμα είναι 0, άρα **Z = 1**.
2	01	OUT	0	0	1	0000	Η θύρα εξόδου παίρνει την τιμή του A: 0 = **0000**.
3	02	LDB 15	0	**15**	1	0000	Βάζει τον αριθμό 15 στον B (μετρητής του βρόχου). Το Z δεν αλλάζει.
4	03	DECB	0	**14**	**0**	0000	B = 15 − 1 = 14. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
5	04	JNZ 03	0	14	0	0000	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
6	03	DECB	0	**13**	0	0000	B = 14 − 1 = 13. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
7	04	JNZ 03	0	13	0	0000	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
… ο βρόχος επαναλαμβάνεται άλλες 11 φορές: ο B κατεβαίνει από 12 έως 2 …							
30	03	DECB	0	**1**	0	0000	B = 2 − 1 = 1. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
31	04	JNZ 03	0	1	0	0000	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
32	03	DECB	0	**0**	**1**	0000	B = 1 − 1 = 0. Ο B έφτασε στο 0, άρα **Z = 1**: ο βρόχος τελείωσε.
33	04	JNZ 03	0	0	1	0000	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (06).
34	06	ADD 1	**1**	0	**0**	0000	A = 0 + 1 = 1. Δεν είναι 0, άρα Z = 0.
35	07	JUMP 01	1	0	0	0000	Πηδάει πάντα στη διεύθυνση 01 (πίσω στην αρχή του βρόχου).
36	01	OUT	1	0	0	**0001**	Η θύρα εξόδου παίρνει την τιμή του A: 1 = **0001**.
37	02	LDB 15	1	**15**	0	0001	Βάζει τον αριθμό 15 στον B (μετρητής του βρόχου). Το Z δεν αλλάζει.
38	03	DECB	1	**14**	0	0001	B = 15 − 1 = 14. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
39	04	JNZ 03	1	14	0	0001	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
40	03	DECB	1	**13**	0	0001	B = 14 − 1 = 13. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
41	04	JNZ 03	1	13	0	0001	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
… ο βρόχος επαναλαμβάνεται άλλες 11 φορές: ο B κατεβαίνει από 12 έως 2 …							

#	PC	Εντολή	A	B	Z	OUT	Τι έγινε
64	03	DECB	1	**1**	0	0001	B = 2 − 1 = 1. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
65	04	JNZ 03	1	1	0	0001	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
66	03	DECB	1	**0**	**1**	0001	B = 1 − 1 = 0. Ο B έφτασε στο 0, άρα **Z = 1**: ο βρόχος τελείωσε.
67	04	JNZ 03	1	0	1	0001	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (06).
68	06	ADD 1	**2**	0	**0**	0001	A = 1 + 1 = 2. Δεν είναι 0, άρα Z = 0.
69	07	JUMP 01	2	0	0	0001	Πηδάει πάντα στη διεύθυνση 01 (πίσω στην αρχή του βρόχου).
70	01	OUT	2	0	0	**0010**	Η θύρα εξόδου παίρνει την τιμή του A: 2 = **0010**.
… το ίδιο επαναλαμβάνεται για τα νούμερα 2, 3, 4 … 14 (βήματα 71 έως 511) …							
512	01	OUT	15	0	0	**1111**	Η θύρα εξόδου παίρνει την τιμή του A: 15 = **1111**.
513	02	LDB 15	15	**15**	0	1111	Βάζει τον αριθμό 15 στον B (μετρητής του βρόχου). Το Z δεν αλλάζει.
514	03	DECB	15	**14**	0	1111	B = 15 − 1 = 14. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
515	04	JNZ 03	15	14	0	1111	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
516	03	DECB	15	**13**	0	1111	B = 14 − 1 = 13. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
517	04	JNZ 03	15	13	0	1111	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
… ο βρόχος επαναλαμβάνεται άλλες 11 φορές: ο B κατεβαίνει από 12 έως 2 …							
540	03	DECB	15	**1**	0	1111	B = 2 − 1 = 1. Δεν είναι 0, άρα Z = 0: ο βρόχος συνεχίζει.
541	04	JNZ 03	15	1	0	1111	Z = 0, άρα **πηδάει** πίσω στη διεύθυνση 03.
542	03	DECB	15	**0**	**1**	1111	B = 1 − 1 = 0. Ο B έφτασε στο 0, άρα **Z = 1**: ο βρόχος τελείωσε.
543	04	JNZ 03	15	0	1	1111	Z = 1, άρα **δεν πηδάει**. Συνεχίζει στην επόμενη εντολή (06).
544	06	ADD 1	**0**	0	1	1111	A = 15 + 1 = 16 → δεν χωράει σε 4 bits, μένει **0** (ξεχείλισμα). Το αποτέλεσμα είναι 0, άρα **Z = 1**.
545	07	JUMP 01	0	0	1	1111	Πηδάει πάντα στη διεύθυνση 01 (πίσω στην αρχή του βρόχου).
546	01	OUT	0	0	1	**0000**	Η θύρα εξόδου παίρνει την τιμή του A: 0 = **0000**.

> **Αποτέλεσμα:** η θύρα εξόδου παίρνει με τη σειρά τις τιμές `0, 1, 2 … 15, 0, 1 …` και το πρόγραμμα δεν σταματάει ποτέ. Κάθε νούμερο μένει στην έξοδο για **50 παλμούς ρολογιού**.

5. Τι αξίζει να προσέξεις

- **Δύο καταχωρητές, δύο δουλειές:** ο A κρατάει το νούμερο που μετράμε και ο B κάνει την καθυστέρηση. Όσο ο B κατεβαίνει, ο A δεν αλλάζει. Με έναν μόνο καταχωρητή αυτό το πρόγραμμα δεν γράφεται.
- **Ο βρόχος καθυστέρησης (`wait`):** το `LDB 15` δίνει 15 επαναλήψεις του `DECB` + `JNZ`. Είναι το `DEC` + `BRNE` του AVR.
- **Το βήμα 34:** το `ADD 1` αλλάζει τον A, αλλά η θύρα εξόδου μένει ίδια. Αλλάζει μόνο στο επόμενο `OUT`, γιατί ο `port_out` είναι καταχωρητής που κρατάει την τιμή του.
- **Το ξεχείλισμα (βήμα 544):** 15 + 1 = 16 δεν χωράει σε 4 bits, οπότε ο A γίνεται 0. Γι' αυτό ο μετρητής ξαναρχίζει μόνος του, χωρίς έλεγχο «έφτασα στο 15;».
- **Γιατί `JUMP loop` και όχι στην αρχή:** το άλμα πάει στο `OUT` (διεύθυνση 01). Αν πήγαινε στο `LOAD 0` (διεύθυνση 00), ο A θα μηδενιζόταν κάθε φορά.
- **Πόσο διαρκεί ένα νούμερο:** OUT 1 + LDB 1 + 15 × DECB 1 + 15 × JNZ 2 + ADD 1 + JUMP 2 = **50 παλμοί**. Οι εντολές άλματος θέλουν 2 παλμούς, γιατί έχουν 2 bytes. Με ρολόι 10 MHz αυτό είναι 5 εκατομμυριοστά του δευτερολέπτου, δηλαδή 200.000 αλλαγές το δευτερόλεπτο. Το μάτι δεν το προλαβαίνει.

- **Για καθυστέρηση που φαίνεται** υπάρχει τώρα ο timer: όλος ο βρόχος `LDB 15` / `DECB` / `JNZ wait` αντικαθίσταται από μία εντολή, `WAIT 13`, που περιμένει 0,42 s στα 10 MHz. Το ίδιο πρόγραμμα με timer είναι το `timer.asm`.